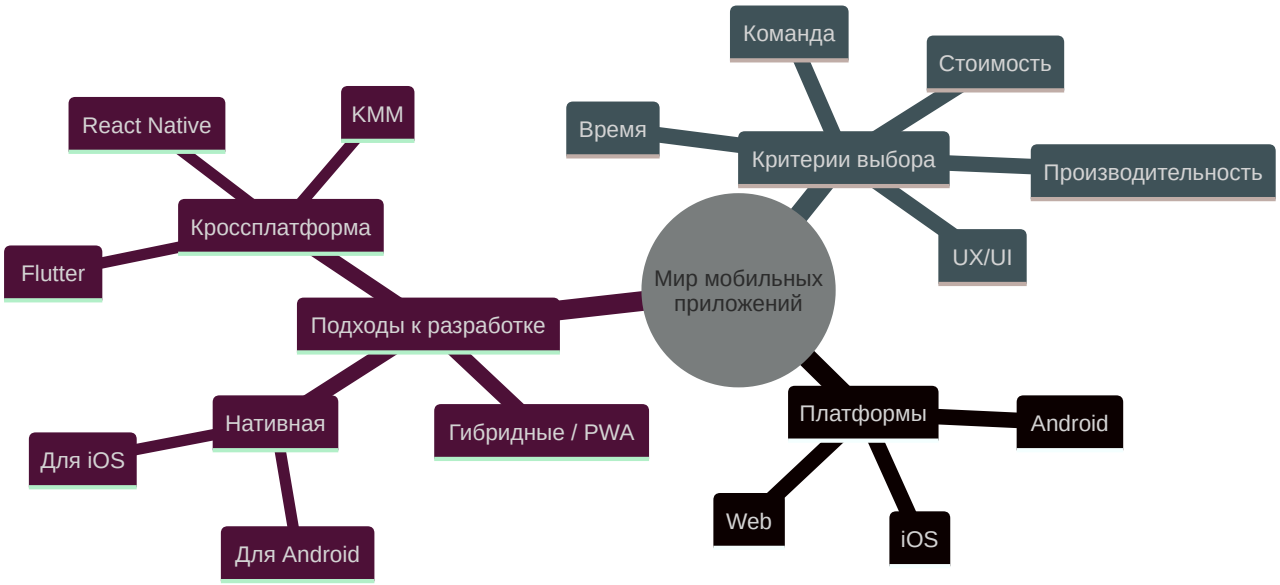
Слайд 1: Введение: мир мобильных приложений

- **Мобильные приложения стали неотъемлемой частью нашей жизни.**
- **Разнообразие платформ:** Android, iOS, и растущий рынок кроссплатформенных решений.
- **Ключевой вопрос для бизнеса и разработчика:** какую стратегию разработки выбрать?
- **Влияние выбора:** на бюджет, время выхода на рынок, производительность и пользовательский опыт (UX).



Заметки к слайду: Сегодня мобильные приложения решают огромный спектр задач: от общения и развлечений до управления бизнесом и сложными финансовыми операциями. Разработчикам и компаниям приходится делать непростой выбор между созданием отдельных приложений для каждой платформы (нативный подход) и использованием технологий, позволяющих писать один код для всех. Этот выбор — стратегическое решение, которое повлияет на весь жизненный цикл продукта. От него зависит, как быстро приложение попадет к пользователям, сколько будет стоить его разработка и поддержка, и насколько качественным будет пользовательский опыт. Мы начнем с обзора этого многообразия, чтобы понять, какие факторы нужно учитывать в первую очередь.

Слайд 2: Цели лекции: стратегии, технологии и выбор платформы

- **Рассмотреть** основные современные подходы к созданию мобильных приложений.
- **Провести** сравнительный анализ нативной и кроссплатформенной разработки.
- **Изучить** архитектуру и ключевые особенности популярных фреймворков.
- **Сформировать** системный подход к выбору технологии на основе конкретных проектных критериев.

Заметки к слайду: Основная цель сегодняшней лекции — дать вам не просто поверхностное знакомство с технологиями, а сформировать у вас целостное понимание ландшафта мобильной разработки. Мы не будем говорить, что один подход лучше другого, потому что это не так. Мы разберем сильные и слабые стороны каждого из них. Вы узнаете, как устроены React Native, Flutter и Kotlin Multiplatform изнутри, и что означает этот выбор для производительности и пользовательского

опыта. В итоге мы выработаем алгоритм, который поможет вам принимать взвешенное решение о выборе стека технологий в зависимости от бюджета, сроков, требований к интерфейсу и специфики задач вашего будущего приложения.

Слайд 3: Классификация подходов к мобильной разработке

- **Нативная разработка:** Создание приложений под конкретную платформу (Android или iOS) с использованием официальных языков, SDK и инструментов.
- **Кроссплатформенная разработка:** Написание единой кодовой базы, которая компилируется или интерпретируется для работы на нескольких платформах.
- **Гибридные приложения и PWA:** Веб-технологии (HTML, CSS, JS), упакованные в оболочку нативного приложения или работающие через браузер, но обладающие некоторыми нативными функциями.

Заметки к слайду: Чтобы структурировать наши знания, мы разделим все существующие подходы на три большие категории. Первая — это нативная разработка, создание приложения "под ключ" для конкретной операционной системы с помощью ее "родных" языков и инструментов. Вторая — кроссплатформенная разработка, где мы пишем общий код на одном языке, а затем получаем приложения для Android и iOS. Сюда входят самые популярные сегодня фреймворки. И третья категория — гибридные приложения и прогрессивные веб-приложения (PWA), которые стоят особняком, используя веб-технологии. На этом слайде представлена ментальная карта, которая поможет нам не запутаться в дальнейшем путешествии по миру мобильной разработки. Каждая из этих ветвей имеет свою философию и идеальные сценарии применения.

Слайд 4: Блок 1: Нативная разработка. Что такое «натив»?

- **«Нативный» (от англ. native — родной)** — значит созданный специально для конкретной операционной системы.
- **Используются** официальные языки программирования, SDK (Software Development Kit) и инструменты разработчика от Apple и Google.
- **Примеры:** Приложение Google Maps для Android написано на Kotlin/Java; Приложение Apple Store для iOS написано на Swift.
- **Результат:** Прямое взаимодействие с API платформы, доступ ко всему аппаратному обеспечению и максимальная производительность.

Заметки к слайду: Итак, начинаем погружение в нативную разработку. Термин "нативный" в контексте IT означает "родной". Нативное мобильное приложение пишется на том языке, который является "родным" для данной платформы. Для Android это Kotlin или Java, а для iOS — Swift или Objective-C. Разработчик использует официальные инструменты, такие как Android Studio и Xcode, и напрямую обращается к возможностям системы через её Software Development Kit. Это позволяет приложению быть максимально интегрированным в экосистему: использовать стандартные жесты, обновляться синхронно с системой и выглядеть так, как ожидает пользователь. Именно такой подход чаще всего выбирают для создания сложных, высоконагруженных приложений, таких как Google Maps, приложения банков или популярные социальные сети.

Слайд 5: Платформа Android: экосистема и языки

- **Экосистема Android:** огромное разнообразие устройств (телефоны, планшеты, ТВ, часы, Auto) от множества производителей.
- **Официальный язык разработки — Kotlin** (с 2019 года).
- **Java** — исторически первый язык, до сих пор используется в легаси-проектах.
- **C++** — для производительных задач (игры, движки) через Android NDK (Native Development Kit).

Заметки к слайду: Говоря о нативной разработке для Android, важно понимать, насколько разнообразна эта экосистема. В отличие от iOS, Android работает на тысячах моделей устройств от разных производителей с разными размерами экранов, вычислительной мощностью и версиями операционной системы. Это накладывает свой отпечаток на процесс разработки. Что касается языков, то сегодня безусловным лидером и стандартом является Kotlin. В 2019 году Google объявил его предпочтительным языком для Android-разработки. Он современен, лаконичен и безопасен. Однако существует огромный пласт приложений, написанных на Java, и знание его все еще необходимо для поддержки старых проектов. Для задач, требующих максимальной производительности, таких как игры или сложная обработка графики, используется C++ через специальный набор инструментов — Native Development Kit (NDK).

Слайд 6: Kotlin vs Java: эволюция языка для Android

- **Java (1995):** Классический, объектно-ориентированный язык с большой историей. Многословный, подвержен ошибкам (NullPointerException).
- **Kotlin (2011, JetBrains):** Современный, лаконичный, функциональный и объектно-ориентированный. Полностью совместим с Java.
- **2017:** Google официально объявляет о поддержке Kotlin на Android.
- **2019:** Kotlin объявлен предпочтительным (первым) языком для Android-разработки.

Заметки к слайду: История языков программирования для Android — это наглядный пример эволюции инструментов под влиянием потребностей разработчиков. Долгое время безраздельно правил Java — мощный, надежный, но достаточно многословный язык, где простое действие могло требовать написания большого объема кода. Кроме того, знаменитые `NullPointerException` были головной болью каждого Java-разработчика. В 2011 году компания JetBrains, создавая свою среду разработки, поняла, что им нужен более удобный язык, и создала Kotlin. Он предложил лаконичность, безопасность (встроенная защита от нулевых ссылок) и полную совместимость с Java. Это позволило внедрять его в проекты постепенно. Как показано на временной шкале, путь Kotlin к статусу первого языка для Android был стремительным: от анонса поддержки в 2017 году до официального признания в 2019. Сегодня Kotlin — это не просто альтернатива, а основной стандарт, на котором строятся все современные приложения.

Слайд 7: Декларативный подход: Jetpack Compose

- **Современный набор инструментов для создания UI (User Interface).**
- **Декларативный подход:** Вы *описываете*, как должен выглядеть интерфейс при каждом состоянии, а фреймворк сам обновляет его при изменении данных.
- **Код на Kotlin:** Интерфейс пишется целиком на Kotlin, без использования XML.
- **Пример:** Простой экран с приветствием.

```
@Composable
fun Greeting(name: String) {
    Column(
        modifier = Modifier.padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(text = "Привет,")
        Text(
            text = name,
            style = MaterialTheme.typography.headlineMedium,
            color = MaterialTheme.colorScheme.primary
        )
```

```
    }
}
```

Заметки к слайду: Jetpack Compose — это революция в создании интерфейсов для Android. Он предлагает декларативный подход, который коренным образом отличается от старого императивного способа. Что это значит на практике? В императивном подходе мы должны были написать алгоритм, как шаг за шагом найти элемент на экране и изменить его текст или цвет при наступлении какого-то события. В декларативном подходе с Compose мы просто говорим: "При таком-то состоянии данных интерфейс должен выглядеть вот так". Когда данные меняются, Compose автоматически и эффективно перерисовывает только те части экрана, которые необходимо. Весь интерфейс описывается с помощью функций `@Composable`, которые пишутся на чистом Kotlin. Это делает код более предсказуемым, простым для понимания и тестирования. Как показано в примере, создание даже небольшого элемента интерфейса становится интуитивно понятным.

Слайд 8: Императивный подход: XML-вёрстка

- **Традиционный способ создания UI в Android (до Compose).**
- **Императивный подход:** Вы описываете статичную структуру UI в XML-файлах, а затем в коде (Activity/Fragment) находите элементы и изменяете их шаг за шагом.
- **Разделение:** Логика на Kotlin/Java, вёрстка в XML.

XML-файл (layout):

```
<LinearLayout>
    <TextView android:id="@+id/textView1" ... />
    <TextView android:id="@+id/textView2" ... />
</LinearLayout>
```

Kotlin-код (Activity):

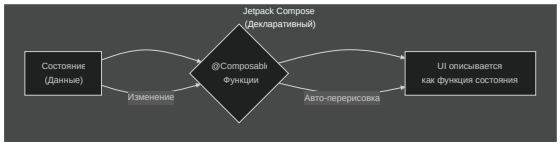
```
override fun onCreate(savedInstanceState: Bundle?) {
    // 1. Загружаем XML
    setContentView(R.layout.activity_main)
    // 2. Находим элементы по ID
    val textView1 = findViewById<TextView>(R.id.textView1)
    val textView2 = findViewById<TextView>(R.id.textView2)
    // 3. Вручную меняем их свойства
    textView1.text = "Привет"
    textView2.text = "Мир"
    textView2.setTextColor(Color.BLUE)
}
```

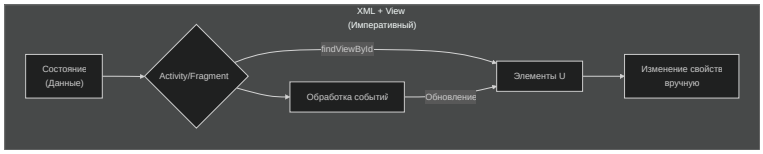
Заметки к слайду: До появления Jetpack Compose разработка интерфейсов в Android была императивной и основывалась на XML-разметке. Процесс выглядел так: сначала дизайнер рисовал экран, потом разработчик создавал XML-файл со статичной структурой: контейнеры, кнопки,

текстовые поля. Затем в коде Activity или Fragment этот XML-файл "надувался" (inflate), и разработчик вручную, с помощью метода `findViewById`, находил каждый элемент, чтобы сохранить на него ссылку. После этого, при наступлении событий или изменении данных, нужно было писать код, который ищет нужный элемент и изменяет его свойство. Этот подход работал, но для сложных динамических интерфейсов он приводил к разрастанию кода, который было трудно поддерживать, и частым ошибкам, особенно при работе с многопоточностью. Это было похоже на инструкцию "взять отвертку, открутить болт А, затем болт Б, надеть новую деталь...".

Слайд 9: Сравнение: Jetpack Compose vs XML

- **Количество кода:** Compose требует значительно меньше кода для реализации одного и того же интерфейса.
- **Управление состоянием:** В Compose состояние — это первостепенная сущность, а UI — его функция. В XML + View состояние управляется вручную.
- **Повторное использование:** В Compose переиспользуются целые `@Composable`-функции, в XML — части разметки через `<include>`.
- **Динамический UI:** В Compose создавать динамические списки и анимации намного проще и интуитивнее.

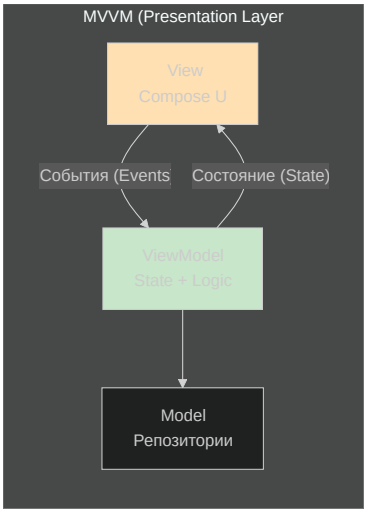


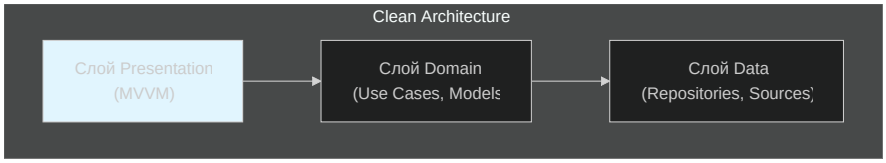


Заметки к слайду: Схема на слайде наглядно демонстрирует ключевое различие между двумя подходами. В императивной модели с XML у нас есть отдельно состояние и отдельно код, который должен "вручную" синхронизировать это состояние с элементами интерфейса. Поток управления сложный и запутанный. В декларативной модели с Jetpack Compose весь UI — это просто функция состояния. Вы объявляете свои composable-функции, которые принимают состояние и отображают его. Когда состояние меняется (например, пользователь нажал кнопку), фреймворк автоматически вызывает эти функции заново, но не перерисовывает все подряд, а эффективно обновляет только изменившиеся части. Это не только сокращает количество кода, но и практически исключает целый класс багов, связанных с рассинхронизацией UI и данных. Jetpack Compose — это шаг в сторону более современной, реактивной и безопасной разработки.

Слайд 10: Архитектура Android-приложения: Clean Architecture / MVVM

- **Clean Architecture:** Разделение кода на независимые слои (Presentation, Domain, Data). Каждый слой решает свою задачу.
- **MVVM (Model-View-ViewModel):** Паттерн проектирования для слоя Presentation, идеально сочетающийся с Jetpack Compose.
    - **View (UI):** Отображает данные и передает действия пользователя. (Compose)
    - **ViewModel:** Хранит состояние и логику для экрана, переживает поворот экрана.
    - **Model:** Слой данных (репозитории, источники данных).





Заметки к слайду: Современная Android-разработка немыслима без грамотной архитектуры. Разработчики давно отошли от практики писать весь код в Activity. Сегодня стандартом де-факто является использование многослойных архитектур, таких как Clean Architecture, в сочетании с паттерном MVVM для пользовательского интерфейса. Clean Architecture предлагает разделить приложение на независимые слои, например: слой данных (Data), отвечающий за получение информации из БД или сети; слой домена (Domain), содержащий бизнес-логику приложения; и слой презентации (Presentation), где живет UI. Это разделение делает код независимым от фреймворков, легко тестируемым и масштабируемым. В слое Presentation используется паттерн MVVM: View (наш Jetpack Compose) подписывается на поток состояний от ViewModel. ViewModel хранит это состояние, обрабатывает действия пользователя и взаимодействует с бизнес-логикой. Такой подход гарантирует, что данные всегда будут отображаться корректно, даже при изменении конфигурации, например, при повороте экрана.

Слайд 11: Ключевые архитектурные компоненты (Android Jetpack)

- **Jetpack** — набор библиотек от Google, которые помогают следовать лучшим практикам, сокращают шаблонный код и упрощают разработку.
- **Основные компоненты для архитектуры:**
  - **ViewModel:** Хранение и управление UI-данными с учётом жизненного цикла.
  - **LiveData / StateFlow:** Наблюдаемые держатели данных, позволяющие UI автоматически реагировать на изменения.
  - **Room:** Библиотека-обёртка над SQLite для удобной работы с базами данных.
  - **Navigation:** Управление навигацией между экранами.
  - **WorkManager:** Планирование отложенных фоновых задач.

Заметки к слайду: Google, понимая сложность создания качественных приложений, разработал набор библиотек под общим названием Android Jetpack. Эти библиотеки призваны решить типовые задачи разработчика, избавить его от написания однотипного кода и помочь создавать приложения, которые будут стабильно работать на разных устройствах и версиях Android. Ключевая особенность Jetpack в том, что библиотеки архитектурно связаны и отлично работают вместе. ViewModel помогает пережить поворот экрана, LiveData или StateFlow доставляют данные в UI в нужный момент, Room обеспечивает типобезопасную работу с базой данных, а Navigation визуализирует и упрощает

перемещение между экранами. WorkManager, в свою очередь, гарантирует выполнение фоновых задач даже при выходе из приложения или перезагрузке устройства. Использование этих компонентов — признак хорошего тона в современной Android-разработке.

Слайд 12: ViewModel и LiveData / StateFlow

- **ViewModel:**
  - Хранит данные, связанные с UI.
  - **Не уничтожается** при повороте экрана или других конфигурационных изменениях.
  - Предоставляет данные для UI и содержит команды для их обновления.
- **LiveData / StateFlow:**
  - **Наблюдаемые** хранилища данных.
  - Позволяют UI (Activity/Fragment/Composable) подписаться на изменения.
  - При изменении данных подписчики автоматически получают уведомления и обновляются.

```
class MyViewModel : ViewModel() {
    // Храним состояние счётчика в StateFlow
    private val _count = MutableStateFlow(0)
    val count: StateFlow<Int> = _count.asStateFlow()

    fun increment() {
        _count.update { it + 1 }
    }
}

@Composable
fun CounterScreen(viewModel: MyViewModel = viewModel()) {
    val count by viewModel.count.collectAsState()

    Column {
        Text("Счётчик: $count")
        Button(onClick = { viewModel.increment() }) {
            Text("Увеличить")
        }
    }
}
```

Заметки к слайду: Рассмотрим работу двух ключевых компонентов Jetpack на практическом примере. ViewModel — это класс, который создаётся один раз для каждого экрана и живёт столько, сколько существует связанный с ним жизненный цикл. Благодаря этому, при повороте экрана данные в ViewModel не теряются, и нам не нужно беспокоиться о их сохранении и восстановлении. Внутри ViewModel мы храним состояние в виде специальных реактивных потоков. Раньше для этого использовали LiveData, но сейчас сообщество и Google всё чаще переходят на Kotlin Flow и его специальную разновидность StateFlow. StateFlow хранит текущее состояние и умеет сообщать о его изменениях. В примере на слайде мы видим, как ViewModel хранит состояние счётчика в `StateFlow<Int>`. Экран на Compose подписывается на этот поток через `collectAsState()`. Как

только пользователь нажимает кнопку и вызывается метод `increment()`, значение в StateFlow меняется, и Compose автоматически перерисовывает экран с новым числом. Это идеальный пример реактивного связывания данных и интерфейса.

Слайд 13: Room Persistence Library

- **Room** — библиотека для работы с локальной базой данных SQLite.
- **Предоставляет слой абстракции** над SQLite, позволяя работать с таблицами как с объектами Kotlin/Java.
- **Компоненты Room:**
  - **Entity** — класс данных, представляющий таблицу в БД.
  - **DAO (Data Access Object)** — интерфейс с методами для доступа к данным (запросы).
  - **Database** — основной класс, связывающий Entity и DAO.

```
// Entity
@Entity(tableName = "users")
data class User(
    @PrimaryKey val id: Int,
    val name: String,
    val age: Int
)

// DAO
@Dao
interface UserDao {
    @Query("SELECT * FROM users")
    fun getAll(): Flow<List<User>>  // Room умеет работать с Flow!

    @Insert
    suspend fun insert(user: User)   // Поддержка корутин
}

// Database
@Database(entities = [User::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun userDao(): UserDao
}
```

Заметки к слайду: Большинству приложений требуется хранить данные локально на устройстве. Будь то кеш ответов от сервера, настройки пользователя или полноценная база данных заметок. Исторически для этого использовался SQLite и сырые SQL-запросы, что было чревато ошибками и требовало много шаблонного кода. Room решает эту проблему элегантно. Он является ORM (Object-Relational Mapping) библиотекой, то есть позволяет работать с таблицами базы данных как с обычными объектами Kotlin. Мы описываем структуру таблицы через класс Entity с аннотациями. Затем создаём интерфейс DAO, где пишем методы для доступа к данным, используя обычные функции Kotlin с аннотированными SQL-запросами. Room на этапе компиляции проверяет корректность этих запросов, что предотвращает многие ошибки ещё до запуска приложения. Особенно важно, что Room идеально интегрирован с современными подходами: он может

возвращать результаты в виде Flow (для реактивного обновления UI) и поддерживает корутины для асинхронных операций.

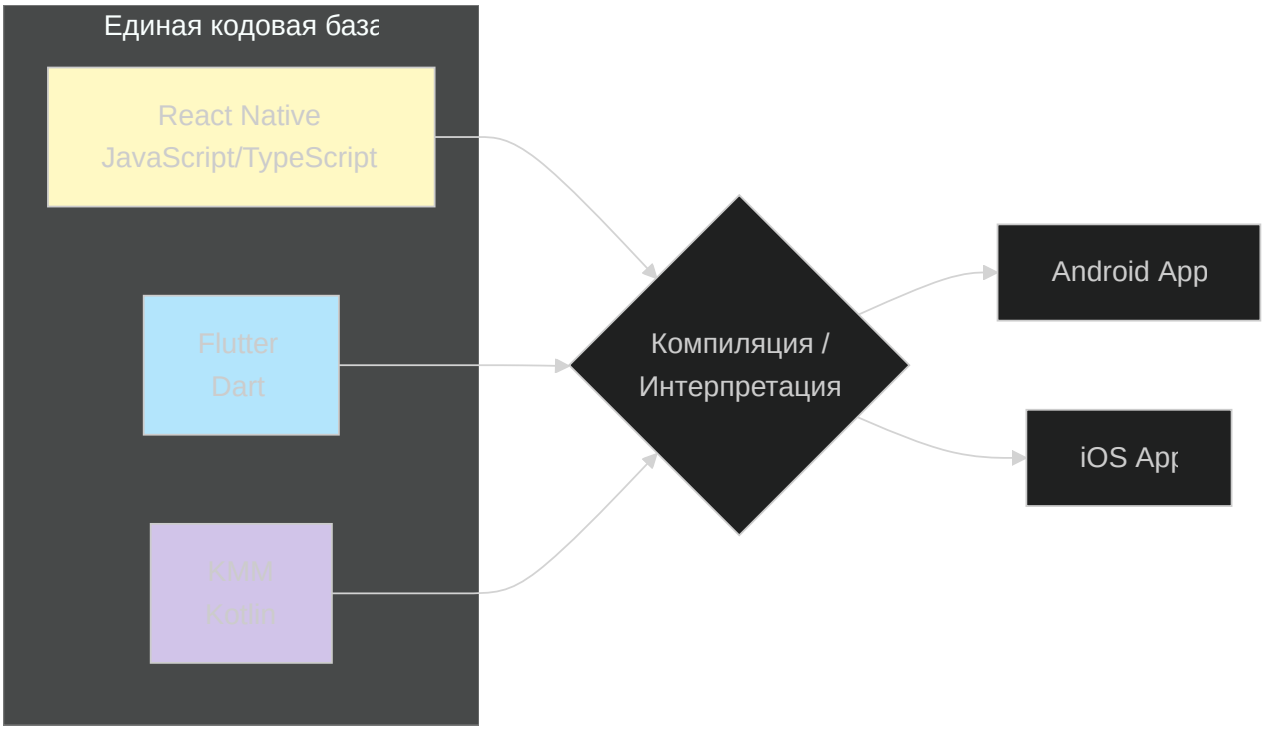
Слайд 14: Преимущества нативной разработки

- 🚀 **Производительность:** Прямой доступ к ресурсам устройства, отсутствие прослоек. Максимальная плавность анимаций (стабильные 60-120 FPS).
- 🔧 **Полный доступ к API:** Все возможности платформы доступны сразу после выхода обновления ОС.
- ✨ **Лучший UX:** Следование гайдлайнам платформы (Material Design для Android) обеспечивает привычный и интуитивный интерфейс для пользователя.
- 🛡️ **Стабильность:** Меньше вероятность ошибок, связанных с фреймворками третьих сторон, проще отладка.

Заметки к слайду: Несмотря на взрывной рост кроссплатформенных технологий, нативная разработка сохраняет свои ключевые преимущества, которые делают её незаменимой в определённых сценариях. Первое и главное — это производительность. Приложение компилируется в машинный код, взаимодействует с системой напрямую, без дополнительных прослоек и мостов. Это гарантирует идеальную плавность анимаций и быстродействие интерфейса. Второе преимущество — доступ ко всем функциям устройства и новым API. Как только Google выпускает новую версию Android с уникальными возможностями, нативные разработчики могут использовать их немедленно. Третье — это пользовательский опыт (UX). Нативное приложение следует гайдлайнам платформы, использует привычные для пользователя жесты и паттерны навигации, что делает взаимодействие с ним интуитивным. И наконец, стабильность: вы не зависите от того, насколько быстро разработчики кроссплатформенного фреймворка исправят баги или добавят поддержку новых функций системы.

Слайд 15: Блок 2: Кроссплатформенная разработка. Философия "Write Once, Run Anywhere"

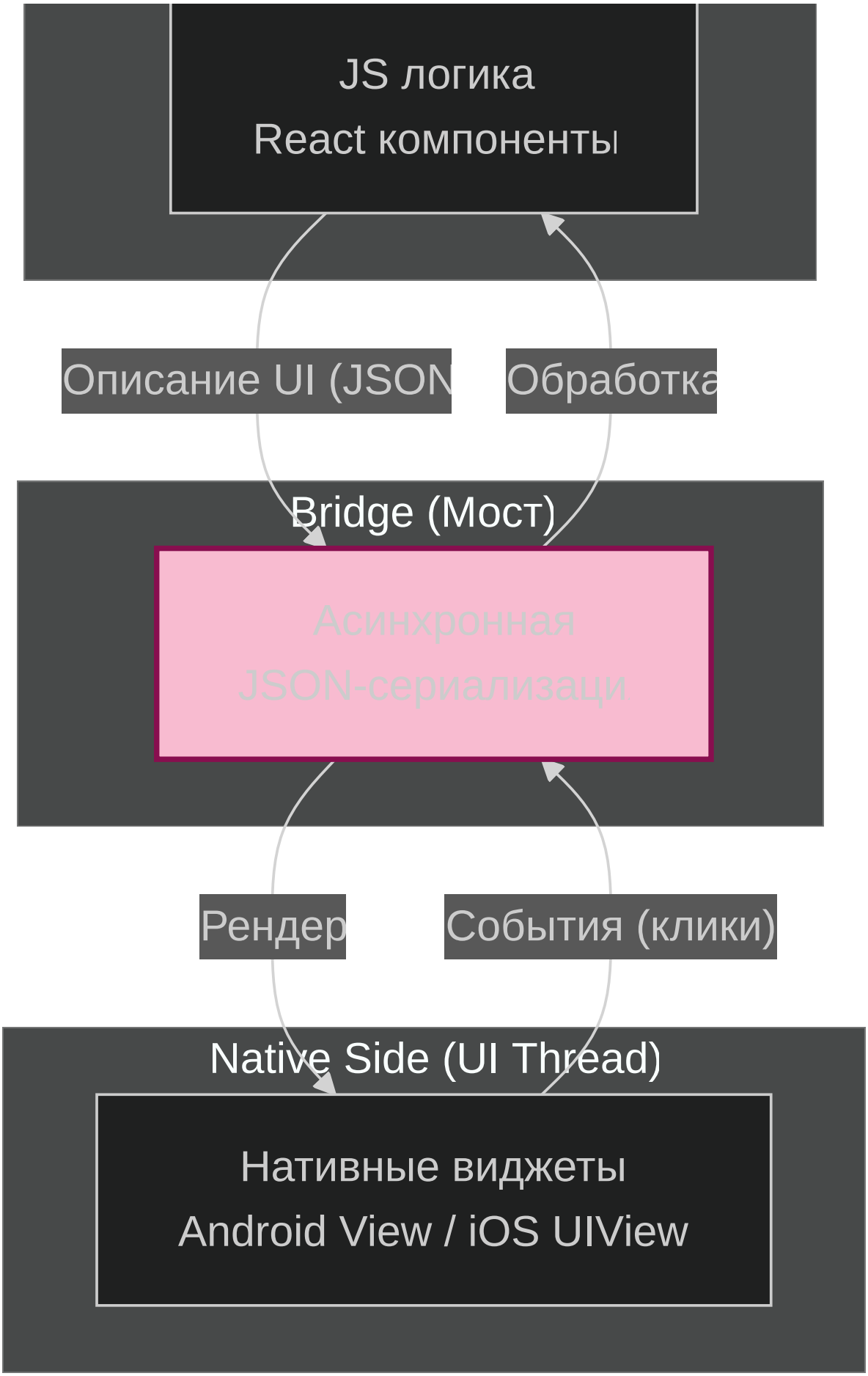
- **Идея:** Написать код один раз и запускать его на нескольких платформах (Android, iOS, а иногда и Web/Desktop).
- **Основная цель:** Сократить время и стоимость разработки, сохранив единую кодовую базу.
- **Компромисс:** Экономия ресурсов может достигаться за счёт производительности, сложности реализации специфических фич или ограничений в доступе к нативным API.



Заметки к слайду: Кроссплатформенная разработка — это ответ на вечный вопрос бизнеса: как охватить максимальную аудиторию (и Android, и iOS) с минимальными затратами? Философия "Write Once, Run Anywhere" (напиши один раз, запускай везде) звучит заманчиво. Вместо двух отдельных команд разработчиков, двух кодовых баз и двух разных графиков релизов, мы получаем одну команду и один проект. Это сулит значительную экономию бюджета и времени выхода на рынок. Однако, как и любой компромисс, кроссплатформа имеет свою цену. Эту цену можно выразить в потенциальных сложностях с производительностью, задержках при доступе к новым функциям ОС, или в необходимости писать нативные "мостики" для сложного оборудования. Нам предстоит разобраться, насколько велика эта цена для каждого из популярных решений. На схеме представлены три главных игрока на этом поле, которые используют совершенно разные подходы к достижению общей цели.

Слайд 16: React Native (Meta): архитектура и мост (Bridge)

- **React Native (RN)** — фреймворк от Meta (Facebook), использующий JavaScript (React) для создания мобильных приложений.
- **Архитектура:** JS-код выполняется в отдельном потоке и общается с нативной частью через асинхронный **"мост" (Bridge)**.
- **Принцип работы:**
    1. JS-поток создаёт описание UI (в виде JSON).
    2. Описание отправляется через мост в нативный поток (сериализуется).
    3. Нативный поток создаёт реальные нативные виджеты (Android View / iOS UIView).
    4. События (клики) отправляются обратно через мост в JS.


JS логика
React компоненты
Описание UI (JSON
Обработка
Bridge (Мост)
Асинхронная
JSON-сериализаци
Рендер
События (клики)
Native Side (UI Thread)
Нативные виджеты
Android View / iOS UIView

Заметки к слайду: Начнём с React Native — одного из самых популярных кроссплатформенных решений. Его ключевая архитектурная особенность — это наличие так называемого "моста" (Bridge). Как работает React Native? Весь ваш код на JavaScript выполняется в отдельном потоке (JS Thread). Когда нужно что-то отрисовать на экране, JS-поток создаёт структурное описание интерфейса в формате JSON и отправляет его через мост в главный (нативный) поток. Мост работает асинхронно, а данные при передаче сериализуются. Нативный поток получает это описание и создаёт реальные нативные компоненты: для Android это обычные View, для iOS — UIView. Когда пользователь нажимает на кнопку, событие точно так же упаковывается в JSON, отправляется через мост обратно в JS, где выполняется соответствующая функция-обработчик. Эта архитектура долгое время была стандартом, но она создаёт потенциальное узкое место — пересылку данных через мост, особенно при большом количестве событий или сложных анимациях.

Слайд 17: React Native: особенности и экосистема

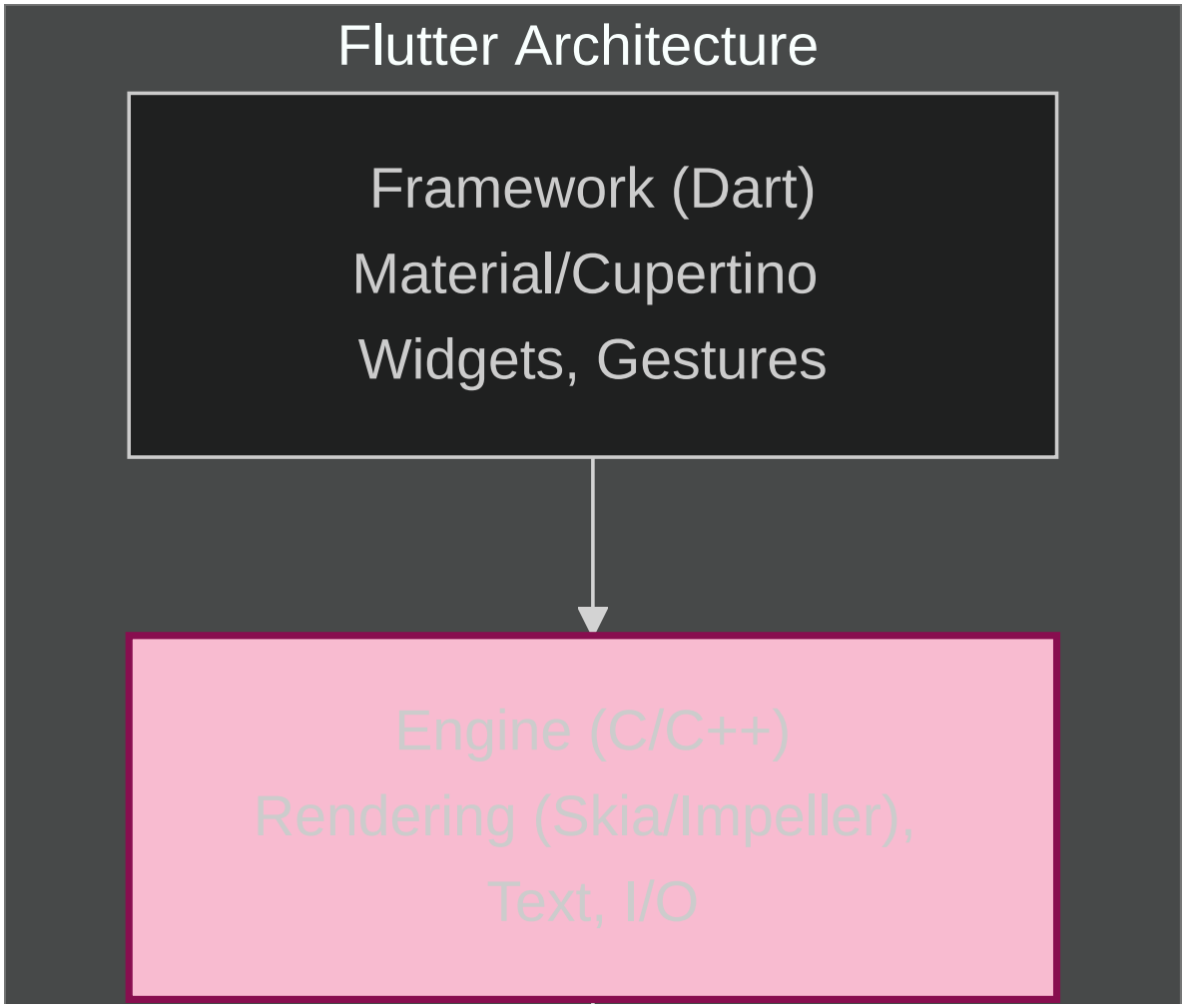
- **Язык:** JavaScript / TypeScript.
- **UI:** Используются нативные компоненты (View, Text, Image), которые внутри маппятся на реальные нативные виджеты.
- **Стилизация:** Через JavaScript-объекты (похоже на CSS-in-JS).
- **Экосистема:**
  - Огромное сообщество и множество готовых библиотек (npm).
  - Возможность горячей перезагрузки (Fast Refresh).
  - Expo — инструмент для упрощения разработки и сборки.
- **React Native —> New Architecture:** Meta активно внедряет новую архитектуру без моста (JSI — JavaScript Interface), позволяющую синхронно вызывать нативные функции и повысить производительность.

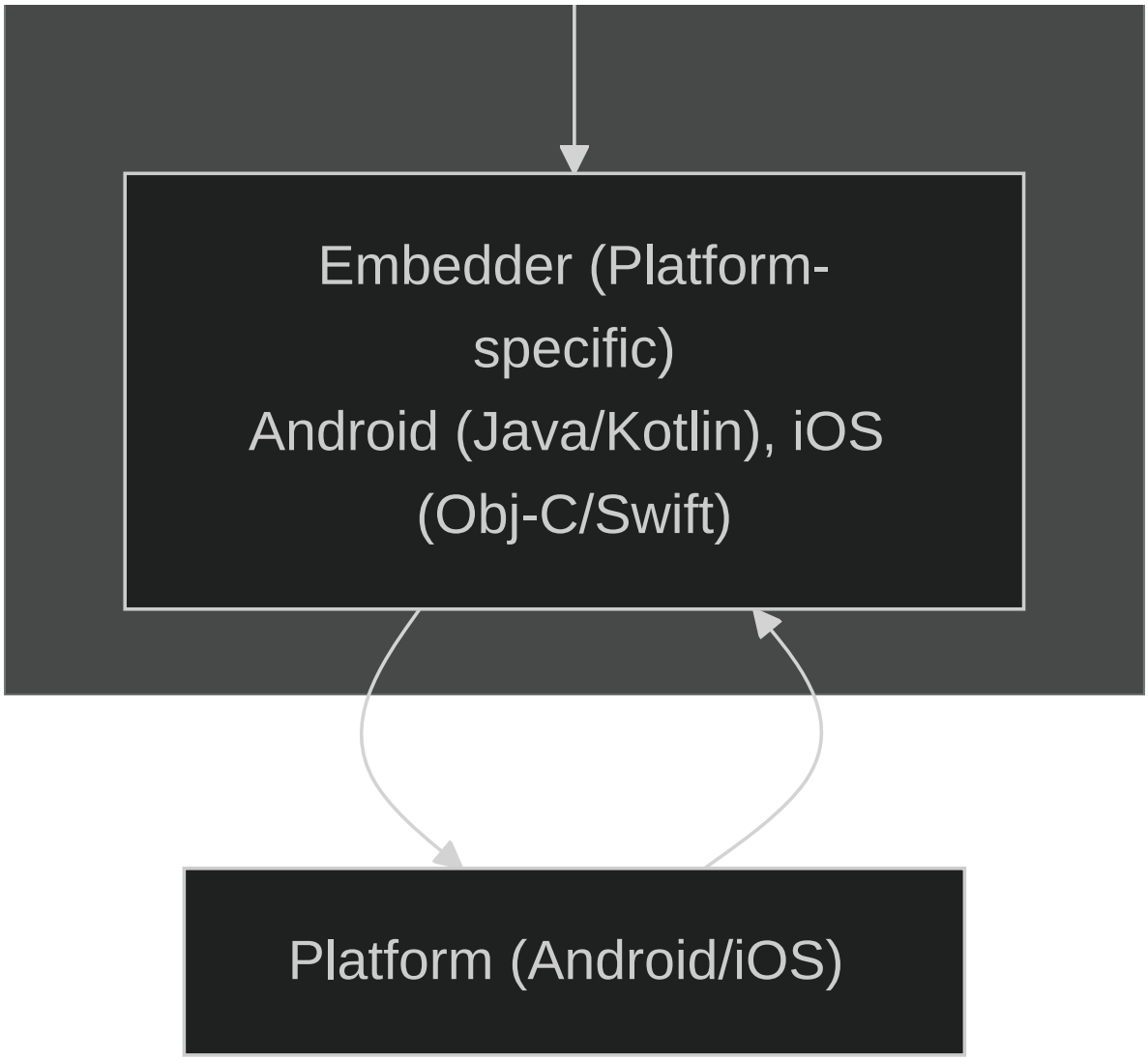
```
import React from 'react';
import { View, Text, Button, StyleSheet } from 'react-native';

const App = () => {
  const [count, setCount] = React.useState(0);

  return (
    <View style={styles.container}>
      <Text style={styles.text}>Вы нажали {count} раз</Text>
      <Button title="Нажми меня" onPress={() => setCount(count + 1)} />
    </View>
  );
};

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', alignItems: 'center' },
  text: { fontSize: 18, marginBottom: 10 }
});

export default App;
```

Заметки к слайду: React Native привлекателен прежде всего для веб-разработчиков, знакомых с React и экосистемой JavaScript. Код выглядит очень знакомо: компоненты, пропсы, состояние (useState), стилизация через объекты, напоминающие CSS. Под капотом компонент `<View>` из React Native превращается в `<android.view.View>` на Android или в `<UIView>` на iOS. Это даёт ощущение работы с нативным UI, но через привычный интерфейс. Огромным плюсом является экосистема: тысячи готовых библиотек на npm решают практически любую задачу. Инструмент Expo позволяет начать разработку без настройки сложных нативных сборок. И, что важно, Meta не стоит на месте: разрабатываемая "Новая архитектура" (New Architecture) с JSI (JavaScript Interface) постепенно отказывается от асинхронного моста в пользу более прямого и синхронного взаимодействия с нативным кодом, что обещает значительный прирост производительности.

Слайд 18: Flutter (Google): архитектура и Render Engine (Skia/Impeller)

- **Flutter** — фреймворк от Google, использующий язык Dart.
- **Уникальность:** Не использует нативные виджеты ОС. Всё рисует сам через собственный **движок рендеринга (Skia / Impeller)**.
- **Архитектура:**
  - **Framework (Dart):** Готовые виджеты, анимации, жесты.
  - **Engine (C++):** Рендеринг (Skia/Impeller), работа с текстом, ввод-вывод.
  - **Embedder (платформо-зависимый код):** Оборачивает Engine в нативное приложение (Android/iOS).





Заметки к слайду: Flutter занимает особое место в мире кроссплатформенной разработки, потому что он предлагает радикально иной подход. Вместо того чтобы транслировать описание интерфейса в нативные виджеты (как React Native), Flutter рисует всё сам. Представьте себе видеоигру: Flutter получает полный контроль над каждым пикселем на экране. В основе этого лежит мощный движок рендеринга — сначала это был Skia (тот же, что используется в браузере Chrome), а теперь Google разрабатывает новый движок Impeller для ещё более плавной работы. Архитектурно Flutter разделён на три уровня. Верхний уровень — фреймворк на Dart, где живут все готовые виджеты. Средний — движок на C++, который отвечает за отрисовку. Нижний — embedder, который фактически "встраивает" это C++-ядро в нативную оболочку для каждой платформы. Такой подход даёт максимальную независимость от платформы и предсказуемость отрисовки.

Слайд 19: Flutter: философия "Everything is a Widget"

- **Всё — виджет.** Экран, кнопка, отступ, выравнивание, анимация — всё это виджеты.
- **Два типа виджетов:**
  - **StatelessWidget:** Статичный, не хранит состояние.
  - **StatefulWidget:** Может хранить состояние и перерисовываться при его изменении.

- **Composition over inheritance:** Сложные UI собираются путём композиции (вложения) простых виджетов.
- **Material и Cupertino:** Встроенные библиотеки виджетов для Android (Material Design) и iOS (Cupertino) стилей.

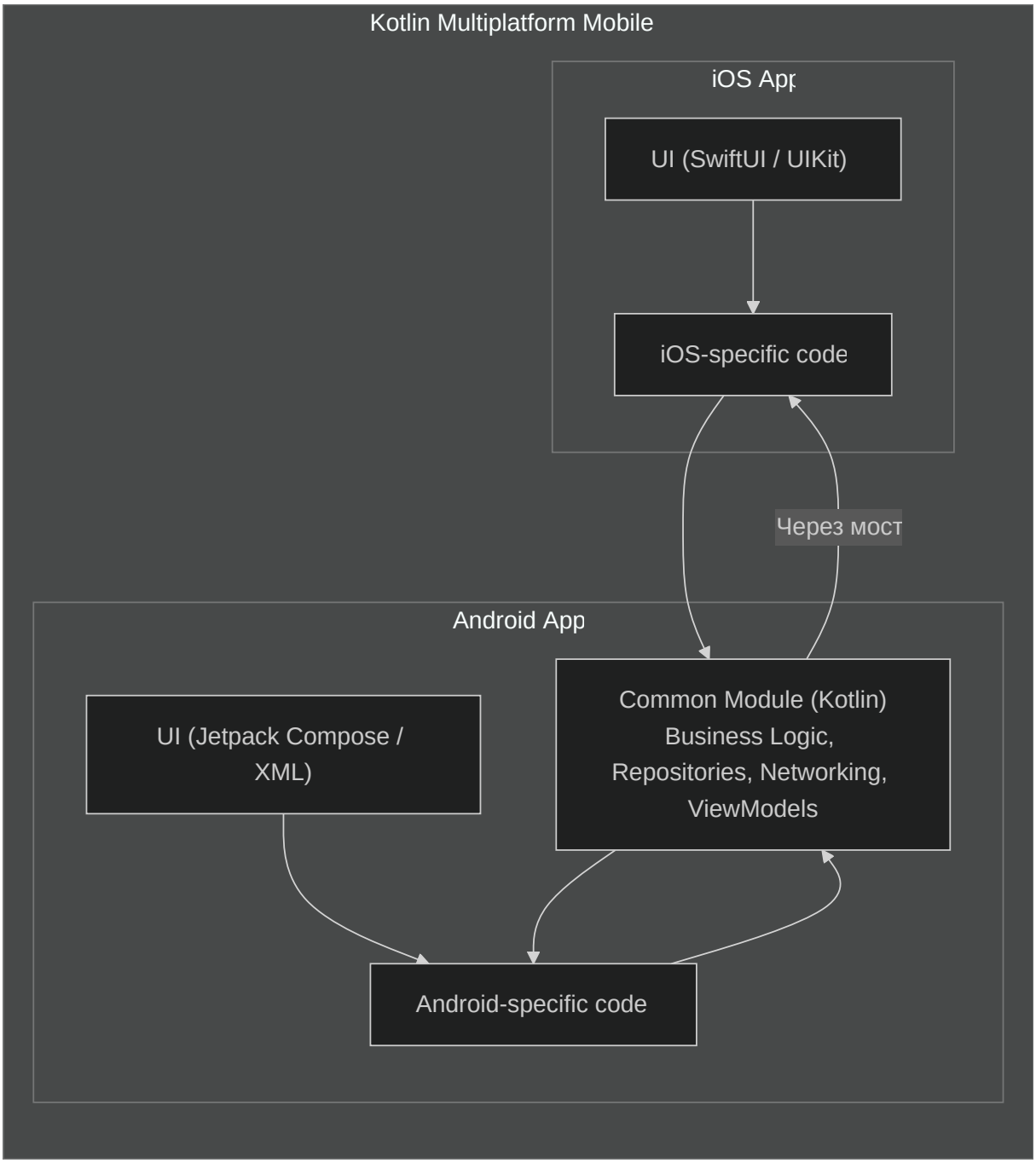
```
import 'package:flutter/material.dart';

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text('Flutter Demo')),
        body: Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Text('Вы нажали 0 раз'),
              ElevatedButton(
                onPressed: () {},
                child: Text('Нажми меня'),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

Заметки к слайду: Философия Flutter проста и элегантна: "Всё есть виджет" (Everything is a Widget). Виджетом является и структурный элемент (кнопка, текст), и стилевой (отступ, выравнивание), и даже само приложение. Вы не просто используете готовые компоненты, а строите иерархическое дерево виджетов, описывая интерфейс. Виджеты делятся на два основных типа: статические (StatelessWidget), которые не меняются со временем, и динамические (StatefulWidget), которые могут обновлять своё содержимое в ответ на действия пользователя или изменения данных. Важно, что Flutter пропагандирует композицию, а не наследование. Вы создаёте сложные интерфейсы, комбинируя и вкладывая друг в друга простые виджеты. Для того чтобы приложения выглядели "родными" на каждой платформе, Flutter предоставляет два набора виджетов: Material (для Android) и Cupertino (для iOS), которые имитируют внешний вид и поведение нативных компонентов.

Слайд 20: Kotlin Multiplatform Mobile (KMM): архитектура

- **KMM (Kotlin Multiplatform Mobile)** — технология от JetBrains, позволяющая использовать Kotlin для написания общей логики для Android и iOS.
- **Философия:** Разделение на **общий код (shared logic)** и **нативный UI**.
- **Архитектура:**

- **Common Module (Kotlin):** Бизнес-логика, работа с сетью, хранение данных, валидация — всё, что не касается UI.
- **Android App:** Нативный UI на Jetpack Compose (или XML) + использование общего модуля.
- **iOS App:** Нативный UI на SwiftUI (или UIKit) + использование общего модуля через Objective-C/Swift мост.



Заметки к слайду: Kotlin Multiplatform Mobile (KMM) занимает уникальную позицию. Это не кроссплатформенный фреймворк в классическом понимании, а скорее инструмент для разделения логики. Философия KMM заключается в том, чтобы писать один раз на Kotlin всё, что не касается интерфейса: сетевые запросы, работа с базой данных, бизнес-процессы, валидация форм. А интерфейс при этом остаётся полностью нативным. Для Android UI пишется на Jetpack Compose (что логично), для iOS — на SwiftUI или UIKit. Общий модуль компилируется в байт-код для Android и в

нативный код для iOS (через промежуточное представление). Это даёт разработчикам лучшее из двух миров: единая и протестированная бизнес-логика и идеальный, нативный пользовательский интерфейс для каждой платформы. KMM — это выбор для проектов, где нативный UX критичен, но хочется избежать дублирования кода логики.

Слайд 21: KMM: разделение логики и UI (Shared Logic)

- **Что выносится в общий код (Shared):**
  - Сетевой слой (Ktor, Retrofit)
  - Репозитории и источники данных
  - База данных (SQLDelight, Room)
  - ViewModel (для Android) и её аналоги для iOS
  - Валидация, форматирование, бизнес-правила
- **Что остаётся нативным:**
  - UI и анимации (Jetpack Compose для Android, SwiftUI для iOS)
  - Навигация
  - Работа с аппаратными функциями (камера, биометрия, Bluetooth) — при необходимости
- **Ожидаемые (expected) и фактические (actual) объявления:** Механизм для реализации платформо-зависимого кода в общем модуле.

```
// В общем модуле (commonMain)
expect class Platform() {
    val platform: String
}

actual class Platform actual constructor() {
    actual val platform: String = "Android
${android.os.Build.VERSION.SDK_INT}"
}

// iOS-модуль
actual class Platform actual constructor() {
    actual val platform: String = "iOS
${UIDevice.currentDevice.systemVersion}"
}
```

Заметки к слайду: Главный вопрос при использовании KMM — что именно следует выносить в общий модуль, а что оставлять платформо-зависимым? Основное правило: в общий код попадает всё, что не связано с отображением интерфейса. Это сетевые запросы, работа с локальным хранилищем, бизнес-логика, ViewModel, содержащая состояние экрана. Таким образом, логика работы приложения становится единой и переиспользуемой. Однако бывают ситуации, когда в общей логике требуется вызвать что-то, что реализовано по-разному на разных платформах (например, получение версии ОС). Для этого в KMM существует механизм `expect`/`actual`. В общем коде мы объявляем ожидаемую функцию или класс с ключевым словом `expect`, а в каждом нативном модуле (Android и iOS) предоставляем его фактическую реализацию с `actual`. Это позволяет сохранить типобезопасность и при этом учесть особенности платформ. Такой подход даёт гибкость нативной разработки с эффективностью повторного использования кода.

Слайд 22: Обзор: Сравнение архитектур RN, Flutter, KMM

Характеристика	React Native	Flutter	KMM
Язык	JavaScript/TypeScript	Dart	Kotlin
Рендеринг UI	Нативные виджеты (через мост)	Собственный движок (Skia)	Нативные виджеты (Android: Jetpack Compose/XML, iOS: SwiftUI/UIKit)
Взаимодействие с нативным кодом	Асинхронный мост (Bridge) / JSI	Platform Channels	Прямой вызов из общего кода (через expect/actual)
UI код	JSX (JavaScript)	Dart-виджеты (декларативно)	Нативный код платформы
Кривая обучения	Низкая (для React-разработчиков)	Средняя (новый язык Dart)	Средняя (для Kotlin-разработчиков, нужно знание iOS)
Размер приложения	Средний	Большой (включает движок)	Малый (почти как нативное)

Заметки к слайду: Давайте систематизируем наши знания и сведём ключевые особенности трёх основных кроссплатформенных подходов в единую таблицу. React Native выбирает путь использования нативных компонентов через мост, что даёт близкий к нативному внешний вид, но может создавать задержки при активном общении между JS и нативным кодом. Flutter идёт другим путём — он рисует всё сам, что обеспечивает максимальную предсказуемость и плавность, но ценой большего размера приложения и иного подхода к созданию UI. KMM стоит особняком: он не пытается создать единый UI, а оставляет его полностью нативным, переиспользуя только логику. Это требует от команды знания обеих платформ (Android и iOS), но даёт наилучший пользовательский опыт и минимальный оверхед. Выбор между ними зависит от приоритетов проекта: скорость разработки, производительность, качество UX или размер команды.

Слайд 23: Блок 3: Глубокое сравнение подходов. Критерий: Время и стоимость

- **Нативная разработка:** Требует двух отдельных команд (Android + iOS) или одной команды, работающей последовательно. **➔ Максимальные бюджет и время.**
- **Кроссплатформенная разработка:** Одна команда пишет общий код для двух платформ. **➔ Экономия бюджета (до ~30-40%) и сокращение time-to-market.**
- **KMM:** Требует специалистов под обе платформы для UI, но экономит на логике. **➔ Средний бюджет, близкий к нативному.**

Заметки к слайду: Один из главных драйверов выбора кроссплатформенной разработки — экономия ресурсов. Давайте оценим это количественно. При нативном подходе вам нужно либо нанимать две отдельные команды (Android и iOS), либо заставлять одну команду делать всё последовательно, что удваивает время разработки. И то, и другое — дорого. Кроссплатформенные фреймворки, такие как React Native и Flutter, позволяют одной команде создавать приложение сразу для двух платформ. Это

даёт ощутимую экономию бюджета (по разным оценкам, от 30% до 50%) и, что критически важно для стартапов, сокращает время выхода на рынок. KMM в этом плане занимает промежуточное положение: вы экономите на написании и поддержке бизнес-логики, но UI-часть всё равно требует нативных специалистов. На графике хорошо видно, где позиционируется каждый подход: React Native и Flutter — в зоне "быстро и дёшево", Натив — "долго и дорого", KMM — ближе к нативу, но с экономией на логике.

Слайд 24: Критерий: Производительность и анимации

- **Натив:** Максимальная производительность, стабильные 60-120 FPS, плавные анимации любой сложности. Полный контроль над каждым кадром.
- **Flutter:** Благодаря собственному движку рендеринга, выдаёт стабильные 60-120 FPS. Анимации не уступают нативным, так как не зависят от "моста".
- **React Native:** Классическая архитектура с мостом может давать просадки FPS при сложных анимациях или большом потоке событий. Новая архитектура (JSI) призвана улучшить ситуацию.
- **KMM:** Производительность UI полностью нативная, так как UI пишется на Jetpack Compose и SwiftUI. Просадок быть не должно.

Заметки к слайду: Производительность и особенно плавность анимаций — это то, что пользователь замечает сразу. Никто не любит "подлагивающие" интерфейсы. На графике представлено сравнение стабильности FPS (кадров в секунду) при выполнении сложных анимаций. Нативные приложения и KMM (где UI нативный) обеспечивают эталонные 60 FPS и выше. Flutter благодаря собственному движку рендеринга, который не зависит от платформенных виджетов, также способен выдавать стабильные 60-120 FPS, что делает его отличным выбором для приложений с богатой графикой. React Native в классической реализации с мостом может испытывать трудности: при интенсивном обмене данными между JS и нативными потоками мост может "забиваться", вызывая пропуск кадров. Однако с переходом на новую архитектуру (JSI) этот разрыв сокращается. Важно понимать, что для большинства бизнес-приложений со стандартными анимациями производительности RN достаточно, но для игр или сложных графических редакторов лучше выбрать натив или Flutter.

Слайд 25: Критерий: Доступ к нативным API и железо

- **Натив:** Полный и мгновенный доступ ко всем API и аппаратным функциям (Bluetooth, NFC, камера, датчики, ARCore/ARKit) сразу после выхода обновления ОС.
- **React Native / Flutter:** Доступ через готовые библиотеки или через написание собственных "нативных мостов" (Native Modules). Задержка в поддержке новых API.
- **KMM:** Доступ к Android API — прямой из Kotlin. Доступ к iOS API — через Objective-C/Swift interoperability. Всё, что не вынесено в общий модуль, делается нативно.

Технология	Доступ к камере	Доступ к Bluetooth	Поддержка новых API
Натив (Android/iOS)	⚡ Прямой	⚡ Прямой	⚡ Мгновенная
Flutter	✅ Через плагины	✅ Через плагины	⏳ С задержкой

Технология	Доступ к камере	Доступ к Bluetooth	Поддержка новых API
React Native	✅ Через библиотеки	✅ Через библиотеки	⏳ С задержкой
KMM	⚡ Прямой (нативный код)	⚡ Прямой (нативный код)	⚡ Мгновенная (для UI)

Заметки к слайду: Если ваше приложение активно использует специфические аппаратные возможности — камеру с нестандартными режимами съёмки, Bluetooth-девайсы, NFC-метки, датчики отпечатков пальцев или технологии дополненной реальности (AR), — к вопросу доступа к нативным API нужно подходить особенно внимательно. В нативном мире проблем нет: вы пишете код, используя официальные SDK, и получаете доступ ко всему немедленно. В мире кроссплатформенных фреймворков всё работает через прослойку. React Native и Flutter предлагают огромный выбор готовых библиотек (плагинов) для типовых задач. Если нужной библиотеки нет, разработчику придётся писать собственный "нативный мост" на Java/Kotlin для Android и на Objective-C/Swift для iOS. Это требует соответствующих знаний и увеличивает сложность. KMM в этом плане гибче: для Android доступ к API осуществляется напрямую из общего Kotlin-кода, а для iOS вы всё равно пишете нативный код (Swift), который может использовать любые API без ограничений.

Слайд 26: Критерий: Кастомизация и сложные UI

- **Натив:** Ограничен только возможностями платформы. Можно реализовать любой дизайн, включая сложные кастомные анимации и переходы.
- **Flutter:** Максимальная гибкость кастомизации благодаря собственному движку рендеринга. Можно нарисовать что угодно, не будучи ограниченным набором нативных компонентов.
- **React Native:** Кастомизация в рамках нативных компонентов. Сложные нестандартные UI могут потребовать написания нативных модулей или использования сторонних библиотек.
- **KMM:** Полная свобода кастомизации на каждой платформе своими средствами (Compose для Android, SwiftUI для iOS).

Заметки к слайду: Представьте, что дизайнер принёс макет с совершенно уникальным интерфейсом, не похожим на стандартные компоненты Material Design или Cupertino. Насколько легко будет его реализовать в разных технологиях? Нативная разработка даёт полную свободу, но реализация сложных кастомных анимаций может потребовать нетривиальных усилий. Неожиданно, но в этой категории Flutter проявляет себя очень сильно. Поскольку он сам рисует каждый пиксель, он не привязан к стандартным виджетам ОС. Вы можете создать интерфейс любой сложности, и он будет одинаково выглядеть на Android и iOS, что иногда является требованием дизайна (например, для фирменного стиля). React Native в большей степени завязан на стандартные нативные компоненты. Создание сильно кастомного UI может быть затруднительным и потребовать либо сложных ухищрений, либо написания нативных частей. KMM, оставляя UI нативным, даёт разработчикам под каждую платформу использовать все возможности Compose и SwiftUI, включая их богатые механизмы кастомизации.

Слайд 27: Критерий: Сообщество, кадры и поддержка

- **React Native:** Крупнейшее сообщество, огромное количество разработчиков (особенно из веба), максимум готовых решений и библиотек.
- **Flutter:** Активно растущее сообщество, сильная поддержка от Google, быстро увеличивающееся количество вакансий и разработчиков.
- **KMM:** Сообщество меньше, но быстро растёт. Требует разработчиков, знающих Kotlin (Android) и, желательно, iOS. Поддержка от JetBrains и Google.
- **Натив:** Огромные сообщества Android (Kotlin/Java) и iOS (Swift) разработчиков по отдельности. Легче всего найти специалиста под конкретную платформу.

Заметки к слайду: При выборе технологии важно учитывать не только технические характеристики, но и "человеческий фактор": легко ли найти разработчиков, как активно развивается фреймворк, много ли готовых библиотек и ответов на Stack Overflow. На графике представлена динамика популярности трёх кроссплатформенных решений за последние 5 лет по данным Google Trends. React Native долгое время был безоговорочным лидером благодаря огромному комьюнити JavaScript-разработчиков. Однако последние годы его популярность немного снижается, хотя он по-прежнему остаётся мейнстримом. Flutter, напротив, демонстрировал взрывной рост и сейчас находится на пике популярности: множество компаний выбирают его, растёт число вакансий. KMM пока уступает по объёму сообщества, но это самый быстрорастущий сегмент: крупные компании (например, Netflix, Square) всё чаще внедряют его в свои проекты. Для найма это означает: React Native — проще всего найти разработчика; Flutter — много специалистов, но спрос тоже высок; KMM — требуются крепкие Kotlin-разработчики с готовностью погружаться в iOS.

Слайд 28: Критерий: Стабильность и обновления

- **Натив:** Максимальная стабильность. Обновления ОС не ломают приложение, если следовать рекомендациям. Полный контроль над версиями SDK.
- **React Native:** Стабильность зависит от версии фреймворка и используемых библиотек. Мажорные обновления React Native иногда требуют значительных усилий по миграции.
- **Flutter:** Высокая стабильность внутри мажорной версии. Google выпускает регулярные обновления. Миграции между версиями обычно хорошо документированы.
- **KMM:** Технология молодая, но активно развивается. Возможны breaking changes. Стабильность UI полностью на плечах нативных разработчиков.

Заметки к слайду: Представьте, что вы выпустили приложение и успешно поддерживаете его. Но проходит год, и выходят новые версии Android и iOS. Как поведёт себя ваше приложение? Насколько легко будет обновить его с учётом изменений в фреймворках? Нативные приложения в этом плане наиболее стабильны: если вы использовали только публичные API, обновление ОС не должно ничего сломать. В мире кроссплатформы ситуация сложнее. React Native и Flutter сами регулярно обновляются, исправляя баги и добавляя функциональность. Однако переход на новую мажорную версию (например, с React Native 0.71 на 0.72) может потребовать времени на адаптацию кода и обновление зависимостей. KMM, как более молодая технология, находится в активной фазе развития, и критические изменения (breaking changes) в ней случаются чаще. Плюс KMM в том, что если "сломается" общая логика, это можно починить в одном месте, а UI, написанный нативно, будет эволюционировать вместе с платформами естественным образом.

Слайд 29: Сводная таблица сравнения

Критерий	Натив (Android)	React Native	Flutter	KMM
Язык	Kotlin	JavaScript/TS	Dart	Kotlin (общий) + Swift
Производительность	☆☆☆☆☆	☆☆☆	☆☆☆☆	☆☆☆☆☆
Качество UI/UX	☆☆☆☆☆	☆☆☆☆	☆☆☆☆	☆☆☆☆☆
Скорость разработки	☆☆	☆☆☆☆☆	☆☆☆☆☆	☆☆☆
Стоимость разработки	Высокая	Средняя	Средняя	Выше средней
Доступ к нативным API	Полный	Через мосты	Через плагины	Полный (на уровне UI)
Размер сообщества	Огромное (по платформам)	Огромное	Большое	Растущее
Time-to-Market	Медленный	Быстрый	Быстрый	Средний
Сложность поддержки	Низкая (раздельно)	Средняя	Средняя	Средняя
Входной порог	Средний (Kotlin)	Низкий (JS/React)	Средний (Dart)	Высокий (Kotlin + iOS)

Заметки к слайду: Перед нами итоговая сводная таблица, которая суммирует всё, что мы обсуждали. Она помогает увидеть картину целиком. Натив — это бескомпромиссное качество и производительность, но ценой ресурсов и времени. React Native — лидер по скорости выхода на рынок и доступности разработчиков, но с потенциальными нюансами производительности. Flutter предлагает отличный баланс производительности и скорости разработки, унифицируя UI. KMM даёт нативное качество интерфейса и переиспользование логики, но требует наиболее квалифицированной команды. Важно понимать, что эта таблица — не истина в последней инстанции, а инструмент для первичной оценки. Конкретные цифры и оценки могут варьироваться в зависимости от специфики проекта, опыта команды и выбранных библиотек. Однако она чётко показывает сильные и слабые стороны каждого подхода.

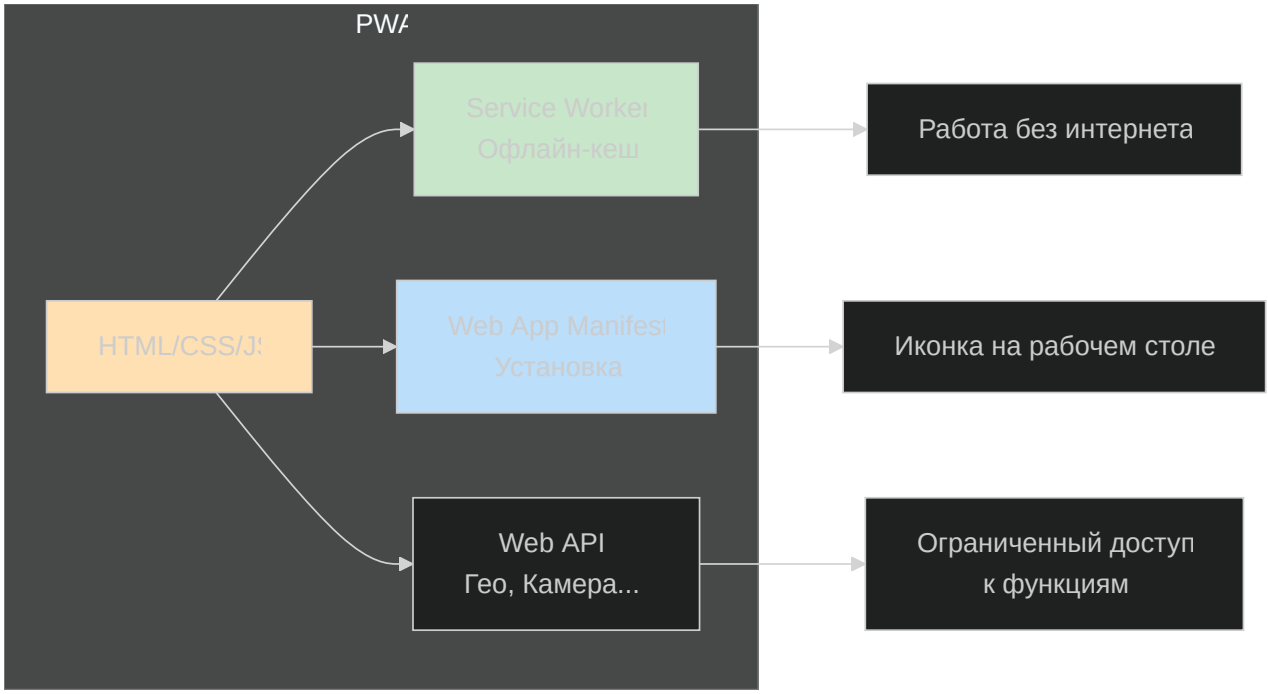
Слайд 30: Блок 4: Гибридные приложения и веб-технологии

- **Отдельная категория** — приложения, построенные на веб-технологиях (HTML, CSS, JavaScript), но упакованные в нативную оболочку.
- **Как работают:** По сути, это веб-страница, открытая внутри нативного компонента **WebView**, который имеет доступ к некоторым аппаратным функциям через плагины.
- **Основные представители:**
    - **Progressive Web Apps (PWA)** — веб-сайты, которые ведут себя как приложения (работа офлайн, push-уведомления, установка на рабочий стол).
    - **Ionic + Capacitor** — фреймворк для создания гибридных приложений с богатой библиотекой UI-компонентов.

Заметки к слайду: Прежде чем перейти к итоговым выводам, стоит упомянуть ещё одну категорию, которая лежит несколько в стороне от основного сравнения — гибридные приложения и PWA. Их ключевое отличие в том, что они используют веб-технологии как основу. Гибридное приложение — это, по сути, сайт, "завёрнутый" в нативную оболочку — WebView. Этот WebView открывается на весь экран и выглядит как обычное приложение, но внутри него работает HTML, CSS и JavaScript. Через специальные плагины (например, в Capacitor) такое приложение может получать доступ к камере, геолокации и другим функциям телефона. Progressive Web App (PWA) — это эволюция веб-сайтов: они могут устанавливаться на устройство, работать без интернета (благодаря Service Workers) и отправлять push-уведомления, но при этом остаются веб-страницами и не проходят через магазины приложений. Это самый лёгкий путь, но и с самыми серьёзными ограничениями по производительности и доступу к желез.

Слайд 31: Progressive Web Apps (PWA): возможности и ограничения

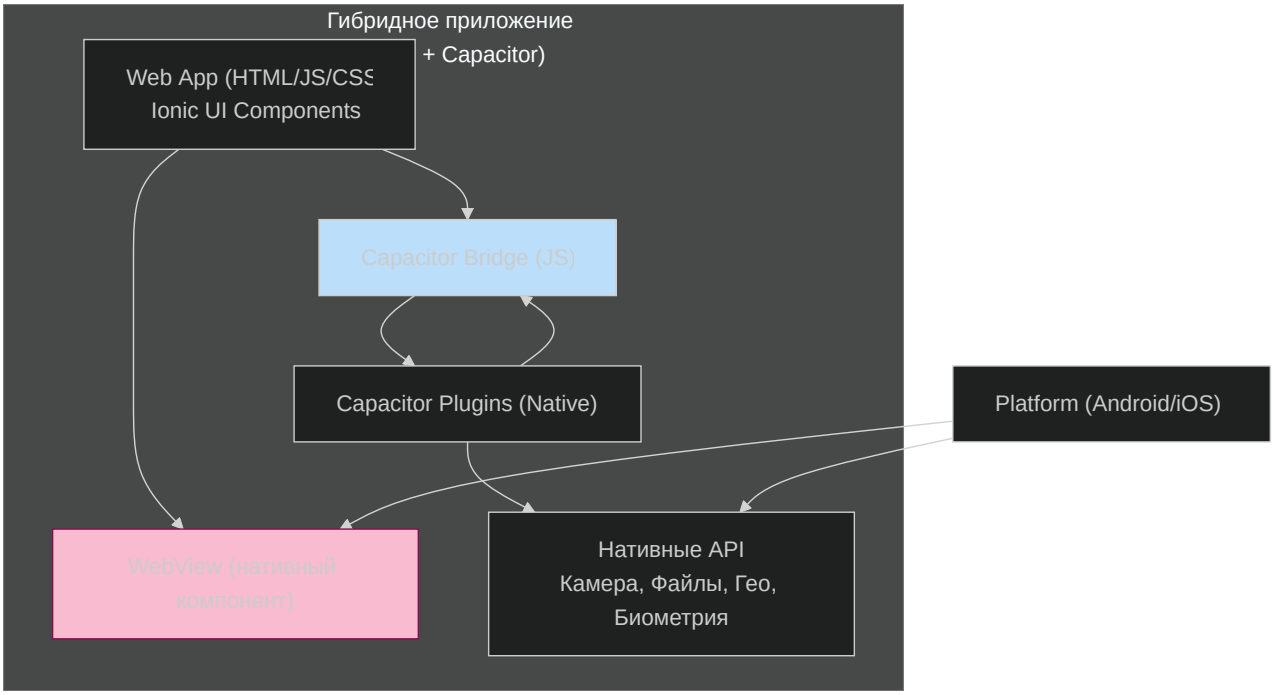
- **PWA** — это веб-приложения, использующие современные возможности браузеров для предоставления пользовательского опыта, близкого к нативным приложениям.
- **Ключевые возможности:**
  - **Установка на устройство** без магазина приложений.
  - **Работа офлайн** (кеширование через Service Workers).
  - **Push-уведомления.**
  - **Доступ к некоторым API** (геолокация, камера, акселерометр).
- **Ограничения:**
  - **Ограниченный доступ к железу** (нет Bluetooth, NFC, фоновых задач).
  - **Производительность ниже нативной** (JavaScript в браузере).
  - **Зависимость от браузера** (разная поддержка API на разных платформах).
  - **Нет доступа к магазинам приложений** (кроме Google Play, где PWA можно упаковать в Trusted Web Activity).



Заметки к слайду: Progressive Web Apps (PWA) занимают уникальную нишу между веб-сайтами и нативными приложениями. Они позволяют бизнесу быстро запустить "приложение" без необходимости проходить модерацию в App Store и Google Play. Пользователь просто заходит на сайт и видит предложение "Добавить на рабочий стол". После установки PWA открывается в собственном окне, без адресной строки браузера. Благодаря Service Workers, такие приложения могут работать офлайн, кешируя контент. Они могут отправлять пуш-уведомления и использовать некоторые API устройства. Однако за простотой распространения скрываются серьёзные ограничения. PWA не имеют доступа к сложному железу: вы не сможете подключиться к Bluetooth-устройству, использовать NFC или работать в фоне длительное время. Производительность также уступает нативу и даже кроссплатформенным фреймворкам, так как код выполняется в браузере. Это идеальный выбор для контентных проектов (новости, блоги, простые лендинги), но не для сложных интерактивных приложений.

Слайд 32: Гибридный подход: Capacitor и Ionic Framework

- **Ionic Framework** — UI-библиотека с готовыми компонентами, стилизованными под нативные (iOS и Android), построенная на веб-технологиях (Angular, React, Vue).
- **Capacitor** — инструмент от создателей Ionic, который "упаковывает" веб-приложение в нативную оболочку и предоставляет мост для доступа к нативным API.
- **Архитектура:**
    1. Веб-приложение (HTML/JS/CSS) работает внутри WebView.
    2. Capacitor предоставляет JavaScript-API для вызова нативных функций.
    3. Нативная часть (плагины) выполняет запрошенные действия и возвращает результат в WebView.



Заметки к слайду: Ionic в сочетании с Capacitor — это классический представитель гибридного подхода. Если вы веб-разработчик и хотите создавать мобильные приложения, используя знакомые фреймворки (Angular, React, Vue), Ionic предлагает богатую библиотеку компонентов, которые

визуально имитируют нативные элементы iOS и Android. Capacitor же решает проблему доступа к функциям устройства. Он действует как мост: из JavaScript-кода вы вызываете метод, например `Capacitor.Plugins.Camera.getPhoto()`. Capacitor передаёт этот вызов в нативную часть, где соответствующий плагин (написанный на Java для Android или Swift для iOS) выполняет действие и возвращает результат обратно в веб-представление. Это позволяет гибридному приложению делать то, что обычный сайт не может — фотографировать, скачивать файлы, сканировать отпечатки пальцев. Однако "бутылочным горлышком" остаётся WebView: вся отрисовка и логика UI по-прежнему выполняются в нём, что ограничивает производительность и плавность, особенно на слабых устройствах.

Слайд 33: Блок 5: Стратегия выбора. Квадрант принятия решений

- **Универсального ответа нет.** Выбор зависит от контекста проекта и приоритетов.
- **Ключевые вопросы, которые нужно задать перед стартом:**
    1. Каков бюджет и сроки?
    2. Насколько сложным должен быть UI/UX? Требуются ли сложные анимации?
    3. Нужен ли глубокий доступ к нативным API (Bluetooth, NFC, AR)?
    4. Какова квалификация текущей команды? Готовы ли учиться новому?
    5. Планируется ли долгосрочная поддержка и развитие?
    6. Нужно ли приложение в магазинах или достаточно PWA?

Заметки к слайду: Мы подошли к самому важному — как на практике принять решение? Секрет в том, что не существует единственно правильного стека технологий. Есть стек, правильный для вашего конкретного проекта. Чтобы не ошибиться, нужно системно подойти к анализу требований. Перед вами квадрант принятия решений, который поможет визуализировать этот процесс. По одной оси мы откладываем потребность в сложных нативных API (работа с Bluetooth, NFC, специфическим оборудованием). По другой — сложность пользовательского интерфейса, требования к анимациям и плавности. В зависимости от того, в какой квадрант попадает ваш проект, предпочтительная технология будет разной. Простые приложения без сложных API можно делать гибридными. Сложные по UI, но без глубокого доступа к железу — отличная задача для Flutter. А если и UI сложный, и API нужны специфические — скорее всего, вам нужен натив или KMM.

Слайд 34: Анализ по критерию: Бюджет и сроки (MVP vs Энтерпрайз)

- **MVP (Minimum Viable Product) / Стартап:**
    - **Цель:** Быстро проверить гипотезу, выйти на рынок, сэкономить.
    - **Рекомендация:** React Native или Flutter. Максимальная скорость разработки, одна команда, покрытие двух платформ.
    - **Альтернатива:** Если бюджет совсем мал и функционал прост — PWA.
- **Энтерпрайз / Крупный проект:**
    - **Цель:** Долгосрочная стабильность, максимальная производительность, безопасность, лёгкость поддержки.
    - **Рекомендация:** Натив (Kotlin + Swift) или KMM (если есть сильная Kotlin-команда и желание переиспользовать логику).
    - **Альтернатива:** Flutter (если UI очень сложный и нужна унификация).

Заметки к слайду: Бюджет и сроки — часто первый и самый важный фильтр при выборе. Здесь нужно чётко разделять два типа проектов: стартапы (или внутренние MVP) и крупные энтерпрайз-системы. Для стартапа время выхода на рынок критично. Вам нужно как можно быстрее показать продукт пользователям и инвесторам. В этой ситуации логично пожертвовать идеальной производительностью ради скорости. React Native или Flutter позволят запуститься на двух платформах за те же деньги и время, что и на одной нативной. Для энтерпрайз-проектов, которые будут жить и развиваться 5-10 лет, приоритеты меняются. На первый план выходят стабильность, безопасность, предсказуемость и лёгкость поддержки. Здесь риск "нарваться" на ограничения фреймворка или проблемы с миграцией может быть неприемлем. Поэтому энтерпрайз чаще выбирает нативную разработку. KMM может быть хорошим компромиссом, если бизнес-логика сложна и её дублирование на двух платформах слишком затратно.

Слайд 35: Анализ по критерию: Требования к UI/UX и анимациям

- **Критически важно:** Приложение должно выглядеть и работать "как родное" на каждой платформе (следовать гайдлайнам Material Design и Human Interface Guidelines).
    - **Выбор:** Натив или KMM.
- **Важно:** Уникальный, сложный, кастомный дизайн, который должен выглядеть одинаково на Android и iOS (фирменный стиль, игры, креативные приложения).
    - **Выбор:** Flutter (благодаря собственному движку рендеринга).
- **Средней сложности:** Типовые экраны, списки, карточки, стандартные анимации.
    - **Выбор:** React Native, Flutter, Натив — все справятся.

Заметки к слайду: Пользовательский интерфейс — это лицо вашего приложения. То, насколько оно приятно и отзывчиво, напрямую влияет на удержание пользователей. При анализе UI/UX требований нужно ответить на вопрос: насколько важно, чтобы приложение выглядело и вело себя в точности как стандартное приложение данной платформы? Если это критично (например, вы делаете приложение для Apple, которое должно строго следовать гайдлайнам Human Interface Guidelines), то выбор очевиден — натив (SwiftUI) или KMM с нативным UI. Если же у вас уникальный дизайн, который должен быть идентичен на обеих платформах (например, фирменное приложение с узнаваемым стилем), то Flutter даёт здесь преимущество: он обеспечит пиксель-перфект идентичность. Для подавляющего большинства бизнес-приложений со стандартными интерфейсами (списки, формы, карточки) подойдёт любой инструмент — вопрос лишь в скорости разработки и квалификации команды.

Слайд 36: Анализ по критерию: Команда и экспертиза

- **У вас есть команда React-разработчиков:**
    - **Лучший выбор:** React Native. Минимальный порог входа, использование существующих знаний.
- **У вас есть команда Kotlin (Android) разработчиков, готовых учиться:**
    - **Лучший выбор:** KMM (логика на Kotlin, нужно будет изучить SwiftUI для UI) или Flutter (новый язык Dart, но единый UI).
- **У вас есть отдельные команды Android и iOS:**
    - **Лучший выбор:** Нативная разработка. Команды работают параллельно, каждая в своей экосистеме. KMM для шаринга логики (если есть воля к внедрению).

- **У вас есть команда из ничего, нужно нанимать:**
    - **Самый простой найм:** React Native (много JS-разработчиков).
    - **Самый быстрорастущий рынок:** Flutter.
    - **Самый сложный найм:** KMM (нужны сильные Kotlin-разработчики с готовностью писать Swift).

Заметки к слайду: Часто выбор технологии предопределён не столько техническими требованиями, сколько человеческим капиталом, который у вас уже есть. Нанимать новую команду с нуля — дорого и рискованно. Если у вас уже есть сильная команда React-разработчиков, пытаться заставить их писать на Swift — безумие. Гораздо эффективнее будет использовать React Native, даже если он не идеален технически. Если у вас есть классная Android-команда на Kotlin, и вы хотите сделать iOS-приложение, самым логичным путём будет либо нанять iOS-разработчиков и делать натив (с синхронизацией через KMM), либо попробовать Flutter, если команда готова выучить Dart. При найме с нуля проще всего найти React Native-разработчиков — это самая многочисленная категория. Flutter-специалистов тоже достаточно, и их число быстро растёт. Специалистов по KMM, которые могут писать и общую логику, и нативные обёртки под iOS, найти сложнее всего, и они, как правило, дороже.

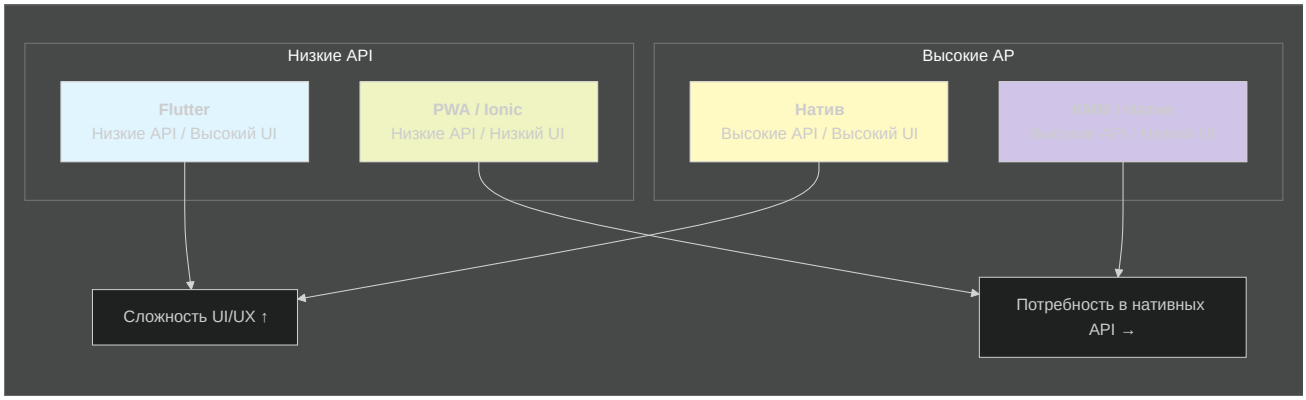
Слайд 37: Анализ по критерию: Специфичные требования (Bluetooth, NFC, AR)

- **Bluetooth Low Energy (BLE), NFC, датчики:** Требуют плотного взаимодействия с нативным кодом.
    - **Натив:** Идеально.
    - **KMM:** Хорошо (нативный UI, прямой доступ из нативного кода).
    - **Flutter/RN:** Возможно, но через плагины. Если нужной библиотеки нет — придётся писать свой нативный мост.
- **Augmented Reality (AR):** ARCore (Android) и ARKit (iOS) — очень разные API.
    - **Натив:** Единственный надёжный путь для сложных AR-сценариев.
    - **Flutter:** Есть плагины (ar_flutter_plugin), но они ограничивают функциональность.
- **Фоновые задачи, синхронизация:**
    - **Натив:** Полный контроль через WorkManager (Android) и BackgroundTasks (iOS).
    - **Кроссплатформа:** Возможны ограничения и различия в поведении.

Заметки к слайду: Этот критерий часто становится "убийцей" кроссплатформенных проектов. Если в техническом задании чётко прописано: "Приложение должно обмениваться данными с Bluetooth-датчиками кардиомонитора" или "Необходимо накладывать 3D-модели на изображение с камеры с помощью ARKit/ARCore", то выбор технологии нужно делать с большой осторожностью. Нативная разработка гарантирует, что вы сможете реализовать любую, даже самую экзотическую функцию, так как у вас есть прямой доступ ко всем API. KMM в этом плане тоже хорош, так как сложные части можно реализовать нативно, а общую логику вынести в shared-модуль. Flutter и React Native полагаются на экосистему плагинов. Для популярных задач (сканирование QR-кодов, геолокация) плагины есть и работают отлично. Но для узкоспециализированных или новых API готового плагина может не быть, и тогда вам придётся нанимать нативных разработчиков для создания этого моста. Это увеличивает стоимость и сложность, сводя на нет преимущества кроссплатформы.

Слайд 38: Итоговая матрица выбора технологии

Потребность в нативных API / Сложность UI	**Низкая**	**Средняя**	**Высокая**
Низкая	PWA / Ionic	React Native	Flutter / Натив
Средняя	React Native	React Native / Flutter	Натив / KMM
Высокая	KMM / Натив	Натив / KMM	**Натив**



Заметки к слайду: Сведём все наши рассуждения в единую матрицу принятия решений. Это практический инструмент, который поможет вам на старте проекта. По горизонтали — потребность в глубокой интеграции с нативными API и специфическим железом. По вертикали — требования к сложности пользовательского интерфейса, анимациям и плавности.

- **Нижний левый угол (всё просто):** Ваше приложение — это, по сути, "обёртка" над сайтом или простой каталог. Выбирайте PWA или Ionic.
- **Верхний левый угол (сложный UI, простые API):** Дизайн — ваша гордость, но приложение не требует сложного доступа к железу. Ваш выбор — Flutter.
- **Нижний правый угол (простой UI, сложные API):** Приложение должно управлять Bluetooth-устройствами, но интерфейс минималистичен. Здесь хорош KMM, позволяющий сделать логику общей, а сложные нативные части — отдельно.
- **Верхний правый угол (всё сложно):** И интерфейс навороченный, и доступ к экзотическим API нужен. Компромиссов быть не может — выбирайте нативную разработку.

Эта матрица не догма, но она помогает структурировать мысли и задать правильные вопросы стейкхолдерам.

Слайд 39: Заключение: Ключевые выводы

- **Нет идеальной технологии.** Каждый подход — набор компромиссов.
- **Натив** — выбор, когда производительность, стабильность и доступ ко всем функциям платформы критичны (и есть бюджет).
- **Flutter** — отличный баланс производительности и единой кодовой базы, особенно для сложных кастомных UI.

- **React Native** — быстрый старт, огромное сообщество, идеально для MVP и проектов с командами из веба.
- **KMM** — будущее для компаний с сильной Kotlin-экспертизой, которые хотят переиспользовать логику, сохраняя нативный UX.
- **Гибриды и PWA** — легковесные решения для простых задач и контентных проектов.
- **Выбор должен быть осознанным,** базироваться на анализе требований, а не на моде или личных предпочтениях.

Заметки к слайду: Мы прошли долгий путь от определения нативной разработки до построения матрицы выбора. Какой главный вывод мы можем сделать? Он прост и сложен одновременно: не существует волшебной палочки. React Native, Flutter, KMM и нативная разработка — это просто инструменты. Молоток не лучше отвертки, а отвертка не лучше пилы. Всё зависит от того, что вы строите и какие у вас плотники. Наша цель как архитекторов и технических лидеров — не слепо следовать трендам, а понимать сильные и слабые стороны каждого инструмента и принимать взвешенное решение, исходя из реальных потребностей бизнеса, пользователей и команды. Где-то нужна скорость выхода на рынок, и React Native будет идеален. Где-то нужна бескомпромиссная производительность сложной графики — и там место для Flutter или натива. А где-то ценно переиспользование кода без потери качества UX — и тогда стоит присмотреться к KMM. Осознанный выбор — залог успеха проекта.

Слайд 40: Список источников

- **Официальная документация:**
  - developer.android.com — Android Developers
  - reactnative.dev — React Native
  - flutter.dev — Flutter
  - kotlinlang.org/docs/multiplatform.html — Kotlin Multiplatform
  - ionicframework.com — Ionic Framework
  - capacitorjs.com — Capacitor
- **Книги и статьи:**
  - "Android Programming: The Big Nerd Ranch Guide"
  - "Flutter in Action" — Eric Windmill
  - "React Native in Action" — Nader Dabit
  - Блоги: Medium (Android Developers, Flutter Community), Habr
- **Доклады с конференций:**
  - Google I/O
  - Meta F8 (Meta Connect)
  - KotlinConf
- **Инструменты для сравнения:**
  - hotframeworks.com
  - stateofjs.com (раздел Mobile)

Заметки к слайду: Для более глубокого погружения в темы, затронутые на лекции, я подготовил список ресурсов. Начинать всегда стоит с официальной документации — это самый актуальный и достоверный источник информации. На Android Developers, reactnative.dev, flutter.dev и сайте Kotlin вы найдёте гайды, API-референсы и примеры кода. Книги из серии "Big Nerd Ranch" и "In Action" отлично подходят для системного изучения. Рекомендую также следить за блогами компаний и

сообществ на Medium и Habr — там часто появляются разборы реальных кейсов и лучшие практики. Доклады с конференций (Google I/O, KotlinConf) позволяют узнавать о новинках из первых рук. И наконец, сайты вроде hotframeworks.com и опросы stateofjs.com помогут отслеживать динамику популярности и востребованности технологий. Спасибо за внимание! Готов ответить на вопросы.